

Finnhub Financial Data ETL Pipeline

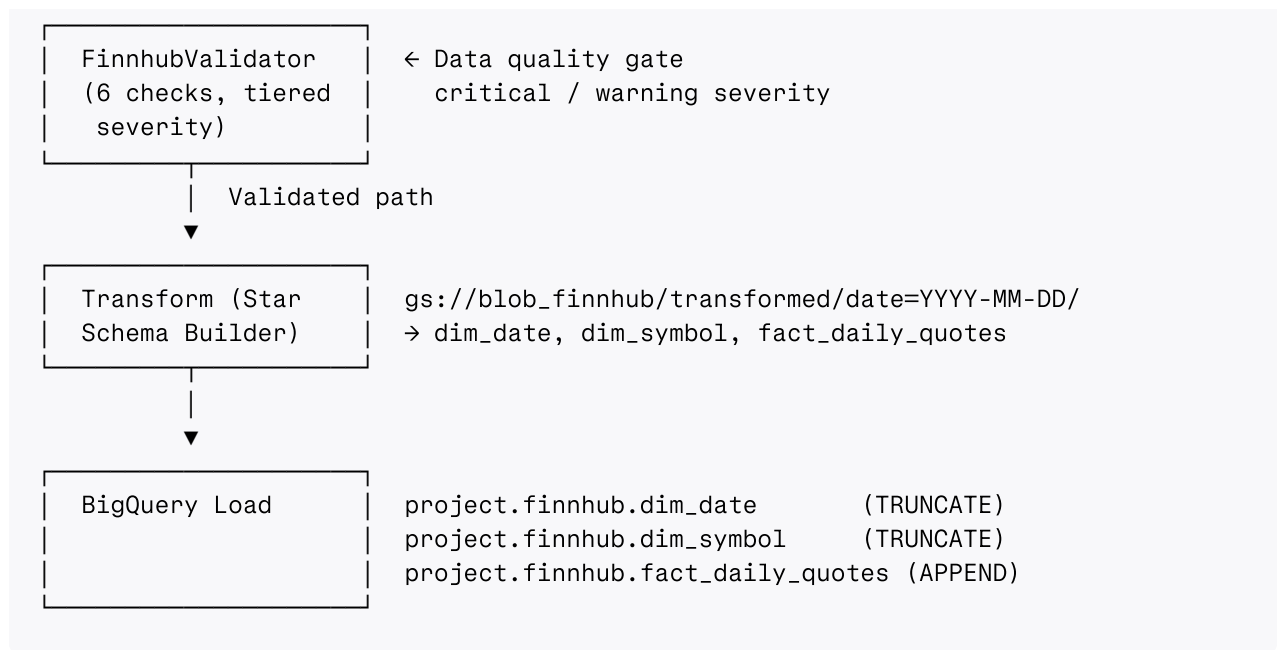
A production-grade, daily financial data pipeline built with **Apache Airflow**, **Google Cloud Storage (GCS)**, and **BigQuery**. The pipeline extracts real-time stock quote data from the [Finnhub API](#), validates it, transforms it into a star schema, and loads it into BigQuery for downstream analytics.

Table of Contents

- [Architecture Overview](#)
- [Pipeline Flow](#)
- [Project Structure](#)
- [Data Model](#)
- [Prerequisites](#)
- [Installation](#)
- [Configuration](#)
- [Airflow Connections](#)
- [Running the Pipeline](#)
- [Validation Rules](#)
- [Testing](#)
- [Known Issues & Roadmap](#)
- [Author](#)

Architecture Overview





Pipeline Flow

The DAG `Finnhub_ETL` runs **daily** (@daily, no catchup) using the **Airflow TaskFlow API**. Tasks are chained as:

```
extract() → validate() → transform() → load()
```

Task	Description
extract	Fetches batch quotes from Finnhub; writes raw Parquet to GCS
validate	Runs 6 data quality checks; fails task on any critical error
transform	Builds <code>dim_date</code> , <code>dim_symbol</code> , <code>fact_daily_quotes</code> ; writes transformed Parquet to GCS
load	Loads all three tables into BigQuery with appropriate write dispositions

On any task failure, a **Slack alert** is sent to the `#Data-Pipeline-Alert` channel via the configured webhook.

Project Structure

```
finnhub-etl/
├── FinnhubConfig.py      # Frozen dataclass configuration for all infra (API, GCS, ...)
├── FinnhubAPI.py         # Custom Airflow BaseHook – session management, retries, ...
├── FinnhubValidator.py   # Data validation layer with structured results and severity
├── FinnhubETL.py         # Airflow DAG definition – 4 TaskFlow tasks (extract → validate → transform → load)
├── test_pipeline.py      # Unit tests for the validator and API hook (pytest)
└── README.md
```

Module Descriptions

FinnhubConfig.py

Defines all infrastructure configuration as **frozen dataclasses** — immutable at runtime. A single PipelineConfig object (pipeline_config) aggregates all sub-configs.

```
pipeline_config.api.rate_limit_per_min # 60
pipeline_config.gcs.bucket_name       # "blob_finnhub"
pipeline_config.bigquery.dataset      # "finnhub"
pipeline_config.alert.slack_channel   # "Data-Pipeline-Alert"
```

FinnhubAPI.py

A custom Airflow BaseHook that:

- Retrieves the API key securely from **Airflow Connections** (no hardcoded secrets)
- Manages a persistent requests.Session with connection pooling
- Implements automatic retries on 429, 500, 502, 503, 504 with exponential backoff (backoff_factor=3)
- Handles 401 (invalid key) as a fatal AirflowException and 404 (unknown symbol) as a graceful skip
- Supports **batch fetching** with configurable rate-limit delay between requests

FinnhubValidator.py

Validates raw DataFrames before transformation. Each check returns a FinnhubValidationResult dataclass with check_name, passed, message, and severity.

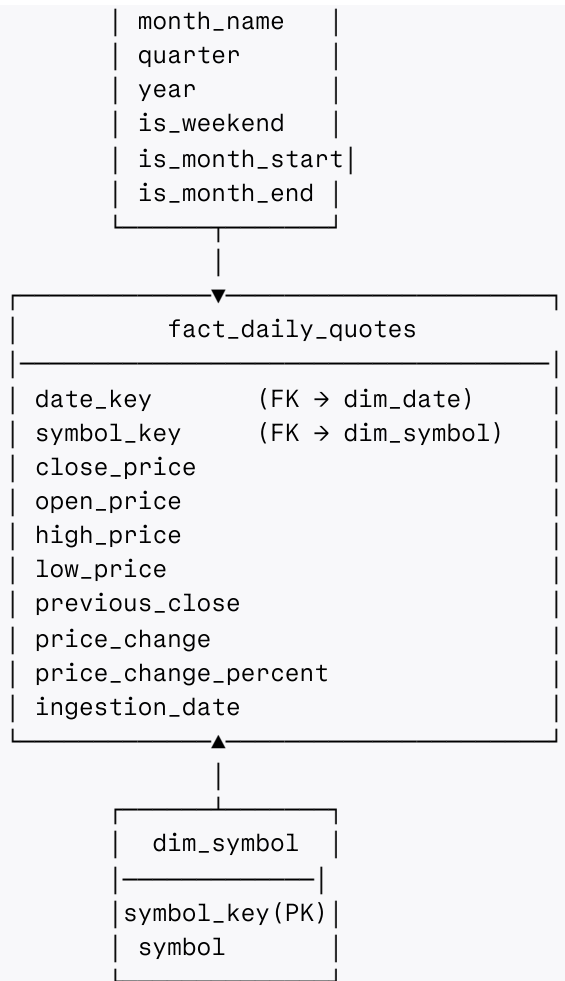
FinnhubETL.py

The main Airflow DAG. Uses the **TaskFlow API** (@task decorator) for clean Python-native task definitions. GCS is used as an intermediate staging layer between tasks, passing file paths through XComs.

Data Model

The pipeline outputs a **star schema** optimised for time-series financial analytics:

dim_date
date_key (PK)
full_date
day_name
day_of_week
week_of_year



Finnhub Quote Fields Mapping

Raw Field	Transformed Column	Description
c	close_price	Current / closing price
o	open_price	Opening price
h	high_price	Daily high
l	low_price	Daily low
pc	previous_close	Previous day's close
d	price_change	Absolute price change
dp	price_change_percent	Percentage price change

Prerequisites

- Python 3.10+
- Apache Airflow 2.11+
- Google Cloud SDK with an active GCP project

- A [Finnhub](#) API key (free tier: 60 requests/min)
- GCS bucket: blob_finnhub
- BigQuery dataset: finnhub

Python Dependencies

```
pip install apache-airflow
pip install apache-airflow-providers-google
pip install apache-airflow-providers-slack
pip install pandas pyarrow google-cloud-storage
pip install pytest pytest-mock # for tests only
```

Installation

1. Clone the repository:

```
git clone https://github.com/your-username/finnhub-etl.git
cd finnhub-etl
```

2. Copy DAG and hook files to your Airflow DAGs folder:

```
cp FinnhubETL.py $AIRFLOW_HOME/dags/
cp FinnhubAPI.py FinnhubConfig.py FinnhubValidator.py $AIRFLOW_HOME/
```

The DAG uses `sys.path.insert(0, "/opt/airflow")` to import the hook and config — ensure all files reside at that path (or update accordingly).

3. Create the GCS bucket and BigQuery dataset (if not already existing):

```
gsutil mb gs://blob_finnhub
bq mk --dataset your-project-id:finnhub
```

Configuration

All settings are managed in `FinnhubConfig.py` via frozen dataclasses. Update the following fields to match your environment:

```
# FinnhubConfig.py

class GCSConfig:
    bucket_name: str = "blob_finnhub" # Your GCS bucket name
    blob_path: str = "blob_finnhub/finnhub_etl"

class BigQueryConfig:
    project: str = "your-gcp-project-id" # ← Replace with your project ID
    dataset: str = "finnhub"

class APIConfigurations:
```

```
rate_limit_per_min: int = 60                # Adjust to your Finnhub plan
# Add default_symbols: frozenset = frozenset({"AAPL", "GOOGL", "MSFT", ...})
```

Important: Add a `default_symbols` field to `APIConfigurations` — this is the list of stock tickers the pipeline will fetch. Without it, the `extract()` task will fail.

Airflow Connections

The pipeline requires two Airflow connections. Configure them via the Airflow UI (Admin → Connections) or the CLI:

1. Finnhub API Key

Field	Value
Connection ID	finnhub_default
Connection Type	HTTP
Password	your_finnhub_api_key

```
airflow connections add finnhub_default \
  --conn-type http \
  --conn-password "your_api_key_here"
```

2. Google Cloud (GCS + BigQuery)

Field	Value
Connection ID	google_cloud_default
Connection Type	Google Cloud
Keyfile Path / JSON	Path to your GCP service account JSON

```
airflow connections add google_cloud_default \
  --conn-type google_cloud_platform \
  --conn-extra '{"key_path": "/path/to/service_account.json", "project": "your-proj"}
```

3. Slack Webhook (Optional — for failure alerts)

Field	Value
Connection ID	slack_webhook
Connection Type	HTTP
Host	https://hooks.slack.com/services/...

Running the Pipeline

1. Start the Airflow scheduler and webserver:

```
airflow scheduler &  
airflow webserver --port 8080
```

2. Unpause the DAG from the Airflow UI or CLI:

```
airflow dags unpause Finnhub_ETL
```

3. Trigger a manual run:

```
airflow dags trigger Finnhub_ETL
```

4. Monitor execution at <http://localhost:8080>.

Validation Rules

The `FinnhubValidator` runs the following checks in order. The pipeline halts on any **critical** failure.

Check	Description	Severity
Not Empty	DataFrame has at least one row	critical
Column Presence	All 7 required columns (c, d, dp, h, l, o, pc) are present	critical
Data Types	All price columns are float64	critical
Null Values	No null values in any column	critical
Price Sanity	Prices are positive; high >= low; prices within a reasonable range	critical
Duplicate Rows	No duplicate rows (keyed on symbol + date if available)	warning

A warning failure is logged but does not block the pipeline from continuing.

Testing

Tests are written with `pytest` and use `pytest-mock` for patching. Run from the project root:

```
pytest test_pipeline.py -v
```

Test Coverage

Test Class	Coverage
TestValidators	<code>_check_not_empty</code> , <code>_check_columns</code> , <code>_duplicate_values</code> , <code>validate_severity_passthrough</code>

Test Class	Coverage
TestFetchData	Successful fetch (200), 404 graceful skip
test_session	Session mock returns correct quote data

Note: A `conftest.py` with a `client` fixture for FinnhubAPI is required for `TestFetchData` and `test_session` to run. See [Known Issues](#).

Known Issues & Roadmap

Current Bugs

Issue	Location	Fix
<code>_read_parquet_from_gcs</code> ignores its parameters; uses hardcoded strings	<code>FinnhubETL.py</code>	Use function parameters instead of hardcoded <code>"blob_finnhub"</code>
<code>extract()</code> iterates dict keys instead of (key, value) pairs	<code>FinnhubETL.py</code>	Change to <code>for symbol, result in hook_results.items()</code>
<code>transform()</code> merges on non-existent <code>ticker/symbol_key</code> columns	<code>FinnhubETL.py</code>	Add surrogate key generation to <code>dim_symbol</code> before merge
<code>EMPTY_QUOTE</code> has an unclosed parenthesis	<code>test_pipeline.py</code>	Add closing <code>)</code>
Import uses wrong filename casing (<code>FinnhubDataValidator</code> , <code>Basehook</code>)	<code>test_pipeline.py</code>	Fix to match actual filenames
<code>default_symbols</code> not defined in <code>APIConfigurations</code>	<code>FinnhubConfig.py</code>	Add <code>default_symbols: frozenset = frozenset({...})</code>
<code>failure_callback</code> only logs; does not send Slack message	<code>FinnhubETL.py</code>	Wire <code>send_slack_notification</code> into the callback

Roadmap

- `[]` Add `conftest.py` with FinnhubAPI client fixture for tests
- `[]` Add `ingestion_timestamp` (UTC) to `fact_daily_quotes`
- `[]` Add `symbol_key` surrogate key generation in `dim_symbol`
- `[]` Convert `FinnhubAPI.py` from Jupyter notebook (`.ipynb`) to a standard `.py` module
- `[]` Add integration test with a mocked GCS / BigQuery backend
- `[]` Extend to additional Finnhub endpoints (candles, financials, news sentiment)
- `[]` Add dbt transformations on top of the BigQuery layer

Author

Shivam Sharma

Data Engineer | MSc Marketing Analytics, University of Stirling

[LinkedIn](#)

Built with Apache Airflow 2.11 · Google Cloud Storage · BigQuery · Finnhub API · Python 3.12